Tipos Abstractos de Datos (TADs)

Gestión de Catálogo de Productos — Fase 2

Clases nuevas: Cola · Pila · Operacion · Sucursal · RedSucursales · CargadorRed · SimuladorTransferencia

Clases actualizadas: Producto (sucursalId, estado) · Catalogo (TablaHash, actualizarStock)

Contenido

1. TAD: Producto (Entidad Base — actualizado Fase 2) ... 3
1.1 Interfaz Pública ... 3
1.2 Implementación Privada ... 3
2. TAD: Catálogo (Orquestador — actualizado Fase 2) ... 3
2.1 Interfaz Pública ... 3
2.2 Implementación Privada ... 4
3. TAD: Cola<T> (Estructura FIFO genérica) ... 4
3.1 Interfaz Pública ... 4
3.2 Implementación Privada ... 4
4. TAD: Pila<T> (Estructura LIFO genérica) ... 5
4.1 Interfaz Pública ... 5
4.2 Implementación Privada ... 5
5. TAD: Operacion (Registro de Historial) ... 5
5.1 Interfaz Pública ... 5
5.2 Implementación Privada ... 5
6. TAD: Sucursal (Nodo de la Red) ... 6
6.1 Interfaz Pública ... 6
6.2 Implementación Privada ... 7
7. TAD: RedSucursales (Grafo Ponderado) ... 7
7.1 Interfaz Pública ... 7
7.2 Implementación Privada ... 8
8. Estructuras auxiliares de RedSucursales ... 8
Conexion (struct) ... 8
ResultadoRuta (struct) ... 8
9. TAD: CargadorRed (Cargador Multi-CSV) ... 8
9.1 Interfaz Pública ... 8
9.2 Implementación Privada ... 8
10. TAD: SimuladorTransferencia (Máquina de Estados Qt) ... 9
10.1 Interfaz Pública ... 9
10.2 Implementación Privada ... 9

1. TAD: Producto (Entidad Base — actualizado Fase 2)

Representa un producto del supermercado. Fase 2 agrega sucursalId y estado para el seguimiento en la red.

1.1 Interfaz Pública

```
    Producto()
    Producto(nombre, codigoBarra, categoria, fechaCaducidad, marca, precio,
stock)
    Producto(nombre, codigoBarra, categoria, fechaCaducidad, marca, precio,
stock,
             sucursalId, estado = "Disponible")     // Constructor Fase 2
    void mostrar() const
    bool operator<(const Producto& otro) const
    bool operator==(const Producto& otro) const
```

1.2 Implementación Privada

Todos los campos son públicos para acceso directo desde las estructuras.

```
    string nombre
    string codigoBarra
    string categoria
    string fechaCaducidad    // ISO: YYYY-MM-DD
    string marca
    double precio
    int    stock
    string sucursalId        // NUEVO Fase 2
    string estado            // "Disponible" | "EnTransito" | "Agotado"
```

2. TAD: Catálogo (Orquestador — actualizado Fase 2)

Coordina 6 estructuras simultáneamente (Fase 2 agrega TablaHash). Garantiza consistencia transaccional con rollback atómico.

2.1 Interfaz Pública

```
    Catalogo()
    ~Catalogo()
    bool agregarProducto(const Producto& producto)
    bool eliminarProducto(nombre, codigoBarra, categoria, fechaCaducidad)
    Producto* buscarPorNombre(const string& nombre)      // AVL O(log n)
    Producto* buscarPorCodigo(const string& codigo)      // Hash O(1)  ←
NUEVO
    void      buscarPorCategoria(const string& categoria)
    void      buscarPorRangoFecha(const string& inicio, const string& fin)
    void      listarPorNombre() const
    void      listarSimple() const
    void      listarOrdenado() const
```

```
    bool      actualizarStock(const string& codigoBarra, int nuevoStock)  //
← NUEVO
    int       contarProductos() const
    bool      estaVacio() const
    ListaSimple*   obtenerListaSimple() const
    ListaOrdenada* obtenerListaOrdenada() const
    ArbolAVL*      obtenerArbolAVL() const
    ArbolB*        obtenerArbolB() const
    ArbolBPlus*    obtenerArbolBPlus() const
    TablaHash*     obtenerTablaHash() const              // ← NUEVO
```

2.2 Implementación Privada

```
    ListaSimple*   listaSimple
    ListaOrdenada* listaOrdenada
    ArbolAVL*      arbolAVL
    ArbolB*        arbolB
    ArbolBPlus*    arbolBPlus
    TablaHash*     tablaHash   // ← NUEVO
    int            totalProductos
    bool existeCodigo(const string& codigoBarra) const
```

3. TAD: Cola<T> (Estructura FIFO genérica)

Cola genérica FIFO con lista enlazada. Cada sucursal posee tres colas: ingreso, traspaso y salida. Al ser un template header-only, no tiene .cpp asociado.

3.1 Interfaz Pública

```
    Cola()
    ~Cola()
    void encolar(const T& dato)          // O(1) — inserta al final
    bool desencolar()                    // O(1) — elimina del frente
    T&         frente()                  // O(1) — acceso al frente
    const T&   frente() const
    bool estaVacia() const               // O(1)
    int  obtenerTamano() const           // O(1)
```

3.2 Implementación Privada

```
    struct NodoCola { T dato; NodoCola* siguiente; }
    NodoCola* cabeza   // frente de la cola (se desencola aquí)
    NodoCola* cola     // final de la cola  (se encola aquí)
    int       tamano
```

4. TAD: Pila<T> (Estructura LIFO genérica)

Pila genérica LIFO con lista enlazada. Cada sucursal posee una pila de operaciones para rollback. Template header-only.

4.1 Interfaz Pública

```
Pila()
~Pila()
void apilar(const T& dato)          // O(1) — inserta en la cima
bool desapilar()                    // O(1) — elimina la cima
T&         cima()                   // O(1) — acceso a la cima
const T&   cima() const
bool estaVacia() const              // O(1)
int  obtenerTamano() const          // O(1)
```

4.2 Implementación Privada

```
struct NodoPila { T dato; NodoPila* siguiente; }
NodoPila* cima_
int       tamano
```

5. TAD: Operacion (Registro de Historial)

Struct que registra una operación realizada sobre el catálogo. Se apila en la pila de historial de cada sucursal para permitir deshacer.

5.1 Interfaz Pública

```
Operacion(Tipo tipo, const Producto& producto,
          const string& sucursalId, const string& sucursalDestino = "")
enum Tipo { AGREGAR, ELIMINAR, TRANSFERIR }
```

5.2 Implementación Privada

```
Tipo        tipo
Producto    producto         // copia del producto afectado
string      sucursalId       // sucursal donde ocurrió
string      sucursalDestino  // solo para TRANSFERIR
```

6. TAD: Sucursal (Nodo de la Red)

Representa una sucursal de la red. Posee un catálogo independiente con 6 estructuras, tres colas de flujo y una pila de historial para rollback.

6.1 Interfaz Pública

```
    Sucursal()
    Sucursal(id, nombre, ubicacion, tiempoIngreso, tiempoTraspaso,
tiempoDespacho)
    ~Sucursal()
```

Getters / Setters:

```
    string getId() const
    string getNombre() const
    string getUbicacion() const
    int getTiempoIngreso() const
    int getTiempoTraspaso() const
    int getTiempoDespacho() const
    Catalogo* getCatalogo() const
    void setNombre(const string& n)
    void setUbicacion(const string& u)
    void setTiempoIngreso(int t)
    void setTiempoTraspaso(int t)
    void setTiempoDespacho(int t)
```

Gestión de inventario:

```
    bool      agregarProducto(const Producto& producto)
    bool      eliminarProducto(nombre, codigo, categoria, fecha)
    Producto* buscarPorNombre(const string& nombre)
    Producto* buscarPorCodigo(const string& codigo)
    int       contarProductos() const
```

Colas de flujo:

```
    void recibirProducto(const Producto& producto)   // → colaIngreso
    void prepararTraspaso(const Producto& producto)  // → colaTraspaso
    void moverATraspasoASalida()                     // colaTraspaso →
colaSalida
    bool despacharSiguiente()                        // pop colaSalida
    Cola<Producto>& getColaIngreso()
    Cola<Producto>& getColaTraspaso()
    Cola<Producto>& getColaSalida()
```

Pila de historial:

```
    void registrarOperacion(const Operacion& op)
    bool deshacerUltimaOperacion()
    bool hayOperacionesPendientes() const
```

```
int  contarOperaciones() const
void mostrar() const
```

6.2 Implementación Privada

```
string          id, nombre, ubicacion
int             tiempoIngreso, tiempoTraspaso, tiempoDespacho
Catalogo*       catalogo
Cola<Producto> colaIngreso, colaTraspaso, colaSalida
Pila<Operacion> pilaOperaciones
```

7. TAD: RedSucursales (Grafo Ponderado)

Grafo ponderado con lista de adyacencia. Cada nodo es una Sucursal y cada arista tiene dos pesos (tiempo y costo). Implementa Dijkstra switcheable por criterio.

7.1 Interfaz Pública

```
RedSucursales()
~RedSucursales()
enum Criterio { TIEMPO, COSTO }
```

Gestión del grafo:

```
bool agregarSucursal(Sucursal* sucursal)                    // O(1)
bool eliminarSucursal(const string& id)                     // O(V+E)
bool agregarConexion(origenId, destinoId, tiempo, costo,
                     bidireccional = true)                  // O(1)
bool eliminarConexion(const string& origenId, const string& destinoId)
```

Consultas:

```
Sucursal*             buscarSucursal(const string& id) const // O(V)
bool                  existeConexion(origenId, destinoId) const
int                   contarSucursales() const
int                   contarConexiones() const
vector<Sucursal*>     obtenerSucursales() const
vector<Conexion>      obtenerConexiones() const
vector<Conexion>      obtenerConexionesDe(const string& id) const
```

Ruta óptima (Dijkstra):

```
ResultadoRuta rutaOptima(origenId, destinoId,
                         criterio = TIEMPO) const           // O((V+E)
log V)
double        calcularETA(const ResultadoRuta& ruta,
                          criterio = TIEMPO) const
bool          transferirProducto(codigoBarra, origenId, destinoId,
                                 criterio = TIEMPO,
                                 unidades = 0)              // 0 = todo
el stock
```

Visualización:

```
void mostrarRed() const
bool generarDot(const string& rutaArchivo) const
bool generarDotConRuta(const string& rutaArchivo,
                       const ResultadoRuta& ruta) const
```

7.2 Implementación Privada

```
struct NodoGrafo { Sucursal* sucursal; vector<Conexion> conexiones; }
vector<NodoGrafo*> nodos
int           indiceDe(const string& id) const             // O(V)
ResultadoRuta dijkstra(origenId, destinoId, criterio) const //
O((V+E)logV)
```

8. Estructuras auxiliares de RedSucursales

Conexion (struct)

```
string origenId, destinoId
double tiempo, costo
bool   bidireccional
```

ResultadoRuta (struct)

```
vector<string> nodos       // IDs de sucursales en orden origen→destino
double         pesoTotal   // suma de pesos de aristas
bool           encontrada  // false si no hay ruta
```

9. TAD: CargadorRed (Cargador Multi-CSV)

Parsea y carga los tres archivos CSV que definen la red. El orden de carga es obligatorio: sucursales → conexiones → productos.

9.1 Interfaz Pública

```
explicit CargadorRed(const string& rutaLog = "data/errors_red.log")
int  cargarSucursales(const string& ruta, RedSucursales& red)
int  cargarConexiones(const string& ruta, RedSucursales& red)
int  cargarProductos (const string& ruta, RedSucursales& red)
bool cargarTodo(rutaSucursales, rutaConexiones, rutaProductos,
                RedSucursales& red)
```

9.2 Implementación Privada

```
string rutaLog
void   loggearError(int linea, const string& motivo,
                    const string& lineaOriginal) const
string limpiarComillas(const string& campo) const
```

```
bool   parsearLinea(const string& linea, string campos[],
                    int numEsperados) const
bool   esEnteroValido(const string& texto) const
bool   esNumeroValido(const string& texto) const
```

10. TAD: SimuladorTransferencia (Máquina de Estados Qt)

Hereda de QObject. Simula el flujo de un producto por la red con QTimer::singleShot encadenado, escalando los tiempos reales por un factor configurable.

10.1 Interfaz Pública

```
enum Etapa { INGRESO, TRASPASO, DESPACHO, EN_VIAJE, ENTREGADO }
explicit SimuladorTransferencia(QObject* parent = nullptr)
void iniciar(red, codigoBarra, origenId, destinoId,
             criterio, unidades, factorAceleracion = 30)
void cancelar()
bool estaActivo() const
```

Señales Qt emitidas durante la simulación:

```
void etapaIniciada(const QString& sucursalId, int etapa, const QString&
mensaje)
void productoEnTransito(const QString& origenId, const QString&
destinoId)
void simulacionCompletada(bool exitosa)
void log(const QString& mensaje)
```

10.2 Implementación Privada

```
RedSucursales*          red
Producto                producto
ResultadoRuta           ruta
int                     unidades, factor, indiceTramo
bool                    activo
QTimer                  timer
RedSucursales::Criterio criterio
void procesarSiguienteEtapa()      // avanza la máquina de estados
void terminarSimulacion(bool exitosa)
void aplicarTransferenciaFinal()   // aplica cambios reales al inventario
int  msEscalado(int segundos) const  // = (segundos*1000) / factor
QString etapaTexto(Etapa e) const
```